

Contents

文字列操作関数リファレンス	4
関数一覧	4
contains	5
starts_with	5
ends_with	5
find	6
substring	6
char_at	6
replace	6
to_upper	6
to_lower	7
trim	7
trim_start	7
trim_end	7
repeat	7
reverse	7
byte_len	8
split	8
join	8
to_int	8
to_float	8
to_str	9
組み込み関数リファレンス	9
関数一覧	9
print	10
Some	11
len	11
has_key	11
add	12
remove	12
range	12
exit	13
args	13
append	13
pop	13
reverse (list)	14
slice	14
filter	14
map	14
sort	14
sort!	15
reverse!	15
appended	15
append!	15
first	15
last	16
get (Map)	16
コレクションリファレンス (タプル・リスト・マップ・セット)	16
タプル	16
リスト	17
マップ	22

セット	23
契約による設計 (Design by Contract)	25
事前条件 (require)	26
事後条件 (ensure)	26
組み合わせ例	26
構造体の不変条件 (invariant)	27
ルール	27
制御構文リファレンス	27
if / elif / else	27
while	28
for	28
break	30
continue	30
... (Ellipsis)	30
match	30
スコープルール	33
ディレクティブ	33
構文	33
対象	33
組み込みディレクティブ	34
注意事項	35
エラーリファレンス	36
エラーフォーマット	36
コンパイルエラー一覧	36
ランタイムエラー一覧	37
関数リファレンス	38
関数定義の構文	38
引数と戻り値の型	38
再帰	38
オーバーロード	39
Unit 型関数	39
ラムダ関数	39
クロージャ	40
関数型	40
高階関数	41
UFCS (統一関数呼び出し構文)	41
演算子オーバーロード	41
演算子リファレンス	42
優先順位テーブル	42
算術演算子	43
比較演算子	43
論理演算子	44
ビット演算子	44
三項条件演算子	44
範囲演算子	45
null 合体演算子 (??)	45
複合代入演算子	45
演算の型規則	45
演算子オーバーロード	46

パッケージリファレンス	47
概要	47
インポート構文	47
パッケージ解決	47
標準ライブラリ (std)	48
検索パスの優先順位	48
RY_PATH 環境変数	48
制約	48
パッケージファイルの作成	49
プロジェクト管理	49
ry init - プロジェクト初期化	49
ry self-update - セルフアップデート	49
ry.toml 設定ファイル	50
正規表現関数リファレンス	51
関数一覧	51
サポートするパターン構文	51
使用例	52
構造体リファレンス	52
概要	52
定義構文	53
コンストラクタ	53
フィールドアクセス	53
フィールド代入	53
関数引数・戻り値としての使用	53
ネスト構造体	54
制約とエラー	54
列挙型 (enum)	54
テスト機能	55
実行方法	56
構文	56
出力形式	57
例	57
モック	57
制限事項	58
型リファレンス	58
型一覧	58
型アノテーション構文	59
使用可能な型名一覧	59
型エイリアス	59
リテラル型	60
範囲型	60
none キーワードと Option 型の省略記法	61
F-String (文字列補間)	61
型キャスト (as)	62
関連データを持つ enum (ADT)	62
ジェネリック enum	63
Error 型	63
union 型	64
型規則 (演算時の型変換)	64
型安全性の制約	65

文字列操作関数リファレンス

文字列 (`str`) に対する操作関数の一覧です。すべての関数で UFCS 記法が使用可能です。

注意: すべての文字列操作は UTF-8 対応です。`len()`、`char_at()`、`substring()`、`find()`、`reverse()` は Unicode コードポイント単位で動作し、バイト単位ではありません。バイト長が必要な場合は `byte_len()` を使用してください。

関数一覧

検索・判定

関数	シグネチャ	説明
<code>contains</code>	<code>(str, str) → bool</code>	部分文字列が含まれるかを返す
<code>starts_with</code>	<code>(str, str) → bool</code>	接頭辞で始まるかを返す
<code>ends_with</code>	<code>(str, str) → bool</code>	接尾辞で終わるかを返す
<code>find</code>	<code>(str, str) → Option<int></code>	部分文字列の文字位置を返す (未発見は <code>None</code>)

抽出・変換

関数	シグネチャ	説明
<code>substring</code>	<code>(str, int, int) → str</code>	部分文字列を取得 (文字インデックス)
<code>char_at</code>	<code>(str, int) → str</code>	指定位置の UTF-8 文字を取得
<code>replace</code>	<code>(str, str, str) → str</code>	部分文字列を全置換

大文字・小文字

関数	シグネチャ	説明
<code>to_upper</code>	<code>str → str</code>	ASCII 大文字に変換
<code>to_lower</code>	<code>str → str</code>	ASCII 小文字に変換

空白除去

関数	シグネチャ	説明
<code>trim</code>	<code>str → str</code>	前後の空白を除去
<code>trim_start</code>	<code>str → str</code>	先頭の空白を除去
<code>trim_end</code>	<code>str → str</code>	末尾の空白を除去

生成・加工

関数	シグネチャ	説明
repeat	(str, int) → str	文字列を n 回繰り返す
reverse	str → str	文字列を逆順にする (UTF-8 対応)
byte_len	str → int	文字列のバイト長を返す

分割・結合

関数	シグネチャ	説明
split	(str, str) → List<str>	デリミタで分割
join	(List<str>, str) → str	セパレータで結合

型変換

関数	シグネチャ	説明
to_int	str → int	文字列を整数に変換
to_float	str → float	文字列を浮動小数点数に変換
to_str	int/float/bool/str → str	値を文字列に変換

contains

シグネチャ: contains(s: str, sub: str) -> bool

文字列 s に部分文字列 sub が含まれるかを返します。

```
print(contains("hello", "ell"))    # true
print("hello".contains("xyz"))     # false (UFCS)
```

starts_with

シグネチャ: starts_with(s: str, prefix: str) -> bool

文字列 s が prefix で始まるかを返します。

```
print(starts_with("hello", "hel"))  # true
print("hello".starts_with("world")) # false (UFCS)
```

ends_with

シグネチャ: ends_with(s: str, suffix: str) -> bool

文字列 s が suffix で終わるかを返します。

```
print(ends_with("hello", "llo"))    # true
print("hello".ends_with("world"))   # false (UFCS)
```

find

シグネチャ: `find(s: str, sub: str) -> Option<int>`

文字列 `s` 中の部分文字列 `sub` の最初の出現位置（文字位置）を返します。見つからない場合は `None` を返します。

```
print(find("hello world", "world"))    # Some(6)
print(find("hello", "xyz"))             # None
print("abcdef".find("cd"))              # Some(2) (UFCS)
```

substring

シグネチャ: `substring(s: str, start: int, end: int) -> str`

文字列 `s` の `start` から `end`（排他）までの部分文字列を返します。インデックスは文字位置（UTF-8 対応）です。

```
print(substring("hello world", 0, 5))   # hello
print(substring("hello world", 6, 11))  # world
print("abcdef".substring(1, 4))         # bcd (UFCS)
```

char_at

シグネチャ: `char_at(s: str, i: int) -> str`

文字列 `s` の `i` 番目の UTF-8 文字を文字列として返します。

```
print(char_at("hello", 0))              # h
print("abc".char_at(2))                 # c (UFCS)
```

replace

シグネチャ: `replace(s: str, old: str, new: str) -> str`

文字列 `s` 中の `old` をすべて `new` に置換した新しい文字列を返します。

```
print(replace("hello world", "world", "ry"))    # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))      # baz bar baz (UFCS)
```

to_upper

シグネチャ: `to_upper(s: str) -> str`

ASCII 小文字（a-z）を大文字に変換した新しい文字列を返します。

```
print(to_upper("hello"))                 # HELLO
print("Hello World".to_upper())          # HELLO WORLD (UFCS)
```

to_lower

シグネチャ: `to_lower(s: str) -> str`

ASCII 大文字 (A-Z) を小文字に変換した新しい文字列を返します。

```
print(to_lower("HELLO"))      # hello
print("Hello World".to_lower()) # hello world (UFCS)
```

trim

シグネチャ: `trim(s: str) -> str`

文字列の前後の空白文字 (スペース、タブ、改行、復帰) を除去した新しい文字列を返します。

```
print(trim(" hello "))      # hello
print(" hi ".trim())        # hi (UFCS)
```

trim_start

シグネチャ: `trim_start(s: str) -> str`

文字列の先頭の空白文字を除去した新しい文字列を返します。

```
print(trim_start(" hello ")) # hello
print(" hi".trim_start())    # hi (UFCS)
```

trim_end

シグネチャ: `trim_end(s: str) -> str`

文字列の末尾の空白文字を除去した新しい文字列を返します。

```
print(trim_end(" hello ")) # hello
print("hi ".trim_end())    # hi (UFCS)
```

repeat

シグネチャ: `repeat(s: str, n: int) -> str`

文字列 `s` を `n` 回繰り返した新しい文字列を返します。

```
print(repeat("ab", 3))      # ababab
print("ha".repeat(3))       # hahaha (UFCS)
```

reverse

シグネチャ: `reverse(s: str) -> str`

文字列を逆順にした新しい文字列を返します (UTF-8 対応)。

```
print(reverse("hello"))     # olleh
print("abc".reverse())      # cba (UFCS)
```

byte_len

シグネチャ: `byte_len(s: str) -> int`

文字列 `s` のバイト長を返します。UTF-8 文字数を返す `len()` とは異なり、`byte_len()` はバイト数を返します。

```
print(byte_len("hello"))    # 5
print(byte_len("あいう"))   # 9
print(len("あいう"))        # 3 (文字数)
```

split

シグネチャ: `split(s: str, delim: str) -> List<str>`

文字列 `s` をデリミタ `delim` で分割し、`List<str>` を返します。

```
let parts = split("a,b,c", ",")
print(parts[0])    # a
print(parts[1])    # b
print(parts[2])    # c

for word in "hello world".split(" "):
    print(word)
# hello
# world
```

join

シグネチャ: `join(xs: List<str>, sep: str) -> str`

文字列リストの要素をセパレータ `sep` で結合した文字列を返します。

```
let parts = ["a", "b", "c"]
print(join(parts, ","))      # a,b,c
print(parts.join("-"))      # a-b-c (UFCS)
```

to_int

シグネチャ: `to_int(s: str) -> int`

文字列を整数に変換します。

```
print(to_int("42"))         # 42
print(to_int("-7"))         # -7
print("123".to_int())       # 123 (UFCS)
```

to_float

シグネチャ: `to_float(s: str) -> float`

文字列を浮動小数点数に変換します。

```
print(to_float("3.14"))     # 3.14
print("2.5".to_float())     # 2.5 (UFCS)
```


to_str

シグネチャ: `to_str(v: int | float | bool | str) -> str`

値を文字列に変換します。

型	変換形式
int	%ld
float	%g
bool	"true" / "false"
str	そのまま返す

```
print(to_str(42))           # 42
print(to_str(3.14))         # 3.14
print(to_str(true))         # true
print(99.to_str())          # 99 (UFCS)
```

[English](#) | [日本語](#) | [繁體中文](#)

組み込み関数リファレンス

関数一覧

コア

関数	説明
<code>print(expr)</code> <code>len(x)</code>	値を標準出力に表示 リスト・マップ・セットの要素数、文字列の UTF-8 文字数を返す
<code>range(n)</code> / <code>range(start, end)</code> / <code>range(start, end, step)</code>	整数のリストを生成
<code>exit(code)</code> <code>args()</code>	指定した終了コードでプロセスを終了 コマンドライン引数を <code>List<str></code> として返す

Option

関数	説明
<code>Some(expr)</code>	<code>Option</code> 型の値ありバリエーションを構築

コレクション操作

関数	説明
<code>has_key(map, key)</code> <code>add(set, value)</code> <code>remove(set, value)</code>	マップにキーが存在するかを返す セットに要素を追加（重複は無視） セットから要素を削除
<code>append(list, value)</code> / <code>append!(list, value)</code> <code>appended(list, value)</code>	リストの末尾に要素を追加（ミューテーション操作） 要素を末尾に追加した新しいリストを返す（非破壊）

関数	説明
<code>pop(list)</code>	リストの末尾の要素を削除して <code>Option<T></code> として返す
<code>reverse(list)</code>	逆順の新しいリストを返す（文字列にも対応）
<code>reverse!(list)</code>	リストをその場で逆順にする（ミューテーション操作）
<code>slice(list, start, end)</code>	<code>start</code> から <code>end</code> までの新しい部分リストを返す
<code>filter(list, pred)</code>	述語を満たす要素だけの新しいリストを返す
<code>map(list, fn)</code>	各要素を変換した新しいリストを返す
<code>sort(list) / sort(list, comp)</code>	ソート済みの新しいリストを返す（デフォルト昇順）
<code>sort!(list) / sort!(list, comp)</code>	リストをその場でソートする（ミューテーション操作）
<code>insert(list, i, val)</code>	インデックス <code>i</code> に要素を挿入
<code>remove_at(list, i)</code>	インデックス <code>i</code> の要素を削除して返す
<code>items(map)</code>	(キー, 値) タプルのリストを返す
<code>remove(map, key)</code>	指定したキーのエントリを削除
<code>get(map, key)</code>	キーの値を <code>Option<V></code> として返す
<code>get(map, key, default)</code>	キーの値を返す（存在しない場合はデフォルト値）
<code>union(set, set)</code>	2つのセットの和集合を返す
<code>intersection(set, set)</code>	2つのセットの積集合を返す
<code>difference(set, set)</code>	2つのセットの差集合を返す
<code>symmetric_difference(set, set)</code>	2つのセットの対称差を返す
<code>is_subset(set, set)</code>	最初のセットが2番目の部分集合かを返す
<code>is_superset(set, set)</code>	最初のセットが2番目の上位集合かを返す

文字列操作

関数	説明
<code>contains(s, sub)</code>	部分文字列が含まれるか
<code>starts_with(s, prefix)</code>	接頭辞で始まるか
<code>ends_with(s, suffix)</code>	接尾辞で終わるか
<code>find(s, sub)</code>	部分文字列の文字位置 (<code>Option<int></code>)
<code>byte_len(s)</code>	文字列のバイト長を返す
<code>substring(s, start, end)</code>	部分文字列を取得
<code>char_at(s, i)</code>	指定位置の文字を取得
<code>replace(s, old, new)</code>	部分文字列を全置換
<code>to_upper(s) / to_lower(s)</code>	大文字・小文字変換
<code>trim(s) / trim_start(s) / trim_end(s)</code>	空白除去
<code>repeat(s, n)</code>	文字列を <code>n</code> 回繰り返す
<code>reverse(s)</code>	文字列を逆順にする
<code>split(s, delim)</code>	文字列を分割してリストを返す
<code>join(list, sep)</code>	リストの文字列をセパレータで結合
<code>to_int(s) / to_float(s) / to_str(v)</code>	型変換

→ 詳細は [文字列操作関数リファレンス](#) を参照

print

シグネチャ: `print(expr)`

値を標準出力に表示します。末尾に改行が付きます。

型	出力形式
int	%ld
float	%g
bool	true / false
str	%s
Option (Some)	Some(値)
Option (None)	None
list	[要素 1, 要素 2, ...]
map	{キー 1: 値 1, キー 2: 値 2, ...}
set	{要素 1, 要素 2, ...}
enum	バリエーション名 (例: Red)

```

print(42)           # 42
print(3.14)         # 3.14
print(true)         # true
print("hello")      # hello
print(Some(1))      # Some(1)
print(None)         # None
print([1, 2, 3])    # [1, 2, 3]
print({"a": 1})     # {a: 1}
print({1, 2, 3})    # {1, 2, 3}

```

エラー条件: 構造体・タプルを直接渡すとコンパイルエラー。

Some

シグネチャ: `Some(expr) -> Option<T>`

Option 型の値ありバリエーションを構築します。

```

let x: Option<int> = Some(42)
print(x)          # Some(42)

```

len

シグネチャ: `len(x: List<T> | Map<K, V> | Set<T> | str) -> int`

リスト・マップ・セットの要素数、または文字列の UTF-8 文字数を返します。バイト長が必要な場合は `byte_len()` を使用してください。

```

print(len([1, 2, 3]))      # 3
print(len({"a": 1, "b": 2})) # 2
print(len({1, 2, 3}))      # 3
print(len("hello"))        # 5
print(len("あいう"))       # 3 (UTF-8 文字数)

```

has_key

シグネチャ: `has_key(m: Map<K, V>, key: K) -> bool`

マップに指定したキーが存在するかを返します。UFCS 記法も使用可能です。

```
let m = {"a": 1, "b": 2}
print(has_key(m, "a"))    # true
print(m.has_key("z"))    # false (UFCS)
```

add

シグネチャ: `add(s: Set<T>, value: T)`

セットに要素を追加します。既に存在する要素を追加した場合は何もしません。UFCS 記法も使用可能です。

```
let s = {1, 2, 3}
s.add(4)          # UFCS
add(s, 5)         # 通常の呼び出し
s.add(1)          # 既に存在するため無視
print(len(s))     # 5
```

remove

シグネチャ: `remove(s: Set<T>, value: T)`

セットから要素を削除します。UFCS 記法も使用可能です。

```
let s = {1, 2, 3}
s.remove(2)       # UFCS
print(2 in s)     # false
```

range

シグネチャ: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

整数のリストを生成します。

形式	生成される値
<code>range(n)</code>	<code>[0, 1, ..., n-1]</code>
<code>range(start, end)</code>	<code>[start, start+1, ..., end-1]</code>
<code>range(start, end, step)</code>	<code>[start, start+step, start+2*step, ...]</code> (end は含まない)

- `step > 0` の場合、`start` から昇順に `end` に向かって生成します。
- `step < 0` の場合、`start` から降順に `end` に向かって生成します。
- `step == 0` の場合、ランタイムエラーになります。

```
print(range(3))          # [0, 1, 2]
print(range(2, 5))       # [2, 3, 4]
print(range(0, 10, 2))    # [0, 2, 4, 6, 8]
print(range(10, 0, -3))   # [10, 7, 4, 1]
```

```
for i in range(3):
    print(i)
# 0
# 1
# 2
```

exit

シグネチャ: `exit(code: int)`

指定した終了コードでプロセスを即座に終了します。`exit()` 以降のコードは到達不能になります。

```
exit(0)      # 正常終了
exit(1)      # エラー終了
```

args

シグネチャ: `args() -> List<str>`

スクリプトに渡されたコマンドライン引数を文字列のリストとして返します。インタプリタ名やスクリプトファイル名は含まれません。スクリプトパスの後の引数のみです。

```
# 実行: ry script.ry hello world
let a = args()
print(len(a))    # 2
print(a[0])      # hello
print(a[1])      # world

for x in args():
    print(x)
```

append

シグネチャ: `append(list: List<T>, value: T)`

リストの末尾に要素を追加します。これはミューテーション操作で、リストがその場で変更されます。UFCS 記法も使用可能です。

```
var xs = [1, 2]
xs.append(3)
print(xs)      # [1, 2, 3]
```

pop

シグネチャ: `pop(list: List<T>) -> Option<T>`

リストの末尾の要素を削除して `Option<T>` として返します。リストが空の場合は `None` を返します。UFCS 記法も使用可能です。

```
var xs = [1, 2, 3]
let v = xs.pop()
print(v)        # Some(3)
print(xs)       # [1, 2]
```

reverse (list)

シグネチャ: `reverse(list: List<T>) -> List<T>`

要素を逆順にした新しいリストを返します。元のリストは変更されません。文字列に対しても使用できます ([文字列操作関数リファレンス](#)を参照)。UFCS 記法も使用可能です。

```
let xs = [1, 2, 3]
let ys = reverse(xs)
print(ys)    # [3, 2, 1]
print(xs)    # [1, 2, 3] (変更なし)
```

slice

シグネチャ: `slice(list: List<T>, start: int, end: int) -> List<T>`

`start` (含む) から `end` (含まない) までの新しい部分リストを返します。インデックスは有効範囲 (0 から `len(list)` まで) にクランプされます。UFCS 記法も使用可能です。

```
let xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (クランプされる)
```

filter

シグネチャ: `filter(list: List<T>, pred: fn(T) -> bool) -> List<T>`

述語が `true` を返す要素のみを含む新しいリストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.filter((x: int) -> x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5] (変更なし)
```

map

シグネチャ: `map(list: List<T>, fn: fn(T) -> U) -> List<U>`

各要素を関数で変換した新しいリストを返します。出力の要素型は入力と異なっても構いません。元のリストは変更されません。UFCS 記法も使用可能です。

```
let xs = [1, 2, 3]
let ys = xs.map((x: int) -> x * 2)
print(ys)    # [2, 4, 6]
```

sort

シグネチャ: `sort(list: List<T>) -> List<T>` / `sort(list: List<T>, comp: fn(T, T) -> bool) -> List<T>`

ソート済みの新しいリストを返します。デフォルトは昇順です。カスタム比較関数を指定できます (第一引数が第二引数の前に来るべき場合に `true` を返す)。元のリストは変更されません。UFCS 記法も使用可能です。

```
let xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# 降順ソート
let desc = xs.sort((a: int, b: int) -> a > b)
print(desc)        # [3, 2, 1]
```

sort!

シグネチャ: `sort!(list: List<T>) / sort!(list: List<T>, comp: fn(T, T) -> bool)`

リストをその場でソートします。ソートアルゴリズムは `sort()` と同じですが、新しいリストを作成する代わりに元のリストを変更します。UFCS 記法も使用可能です。

```
var xs = [3, 1, 2]
xs.sort!()
print(xs)    # [1, 2, 3]
```

reverse!

シグネチャ: `reverse!(list: List<T>)`

リストをその場で逆順にします。UFCS 記法も使用可能です。

```
var xs = [1, 2, 3]
xs.reverse!()
print(xs)    # [3, 2, 1]
```

appended

シグネチャ: `appended(list: List<T>, value: T) -> List<T>`

要素を末尾に追加した新しいリストを返します。元のリストは変更されません。UFCS 記法も使用可能です。

```
let xs = [1, 2]
let ys = xs.appended(3)
print(xs)    # [1, 2] (変更なし)
print(ys)    # [1, 2, 3]
```

append!

シグネチャ: `append!(list: List<T>, value: T)`

`append()` のエイリアスです。リストの末尾に要素をその場で追加します。! 命名規約との一貫性のために提供されています。

first

シグネチャ: `first(list: List<T>) -> Option<T>`

リストの最初の要素を `Option<T>` として返します。リストが空の場合は `None` を返します。

```
print(first([10, 20, 30])) # Some(10)
```

last

シグネチャ: `last(list: List<T>) -> Option<T>`

リストの最後の要素を `Option<T>` として返します。リストが空の場合は `None` を返します。

```
print(last([10, 20, 30])) # Some(30)
```

get (Map)

シグネチャ: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

2 引数形式はキーの値を `Option<V>` として返します。3 引数形式はキーの値またはデフォルト値を返します。

```
let m = {"a": 1, "b": 2}
print(get(m, "a"))      # Some(1)
print(get(m, "z"))      # None
print(get(m, "z", 0))   # 0
```

[English](#) | [日本語](#) | [繁體中文](#)

コレクションリファレンス (タプル・リスト・マップ・セット)

タプル

概要

固定長・異種型の値の組み合わせ。LLVM literal StructType として実装されたスタック上の値型です。

構文

```
let t = (1, 3.14)
let t: (int, float) = (1, 3.14)
```

型アノテーション

```
let pair: (int, str) = (42, "hello")
let triple: (int, float, bool) = (1, 2.0, true)
```

要素アクセス

`.0`, `.1`, ... の数値インデックスでアクセスします。

```
let t = (10, 3.14)
print(t.0) # 10
print(t.1) # 3.14
```

関数戻り値

```
fn swap(a: int, b: int) -> (int, int):
  return (b, a)
```

```
let result = swap(1, 2)
```



```
print(result.0)    # 2
print(result.1)    # 1
```

制約とエラー

制約	詳細
範囲外インデックス	コンパイルエラー
print にタプルを直接渡す	コンパイルエラー (print 非対応)

リスト

概要

同一型の可変長シーケンス。ヒープ上に確保されます。

構文

```
let xs = [1, 2, 3]
let xs: List<int> = [1, 2, 3]
```

対応する要素型

int, float, bool, str

インデックスアクセス

```
let xs = [1, 2, 3]
print(xs[0])    # 1
print(xs[2])    # 3
```

インデックス代入

```
let xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
```

len

```
let xs = [1, 2, 3]
print(len(xs))  # 3
```

print

```
let xs = [1, 2, 3]
print(xs)       # [1, 2, 3]
```

for 走査

```
let xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

リストの末尾に要素を追加します。これはミューテーション操作で、リストがその場で変更されます。

```
var xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

リストの末尾の要素を削除して返します。空のリストに対して呼び出すとランタイムエラーになります。

```
var xs = [1, 2, 3]
let v = xs.pop()
print(v)     # 3
print(xs)    # [1, 2]
```

reverse

要素を逆順にした新しいリストを返します。元のリストは変更されません。文字列に対しても使用できます。

```
let xs = [1, 2, 3]
print(reverse(xs))    # [3, 2, 1]
print(xs)             # [1, 2, 3] (変更なし)
```

slice

start (含む) から end (含まない) までの新しい部分リストを返します。インデックスは有効範囲にクランプされます。

```
let xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (クランプされる)
```

filter

述語を満たす要素だけを含む新しいリストを返します。元のリストは変更されません。

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.filter(fn(x: int): x > 3)
print(ys)    # [4, 5]
```

map

各要素を関数で変換した新しいリストを返します。出力の要素型は入力と異なっても構いません。元のリストは変更されません。

```
let xs = [1, 2, 3]
let ys = xs.map(fn(x: int): x * 2)
print(ys)    # [2, 4, 6]
```

sort

ソート済みの新しいリストを返します。デフォルトは昇順です。カスタム比較関数を指定できます。元のリストは変更されません。

```
let xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]
```

降順ソート

```
let desc = xs.sort(fn(a: int, b: int): a > b)
print(desc)    # [3, 2, 1]
```

filter, map, sort のチェーン

これらの関数は新しいリストを返すため、UFCS で連鎖できます。

```
let xs = [5, 3, 1, 4, 2]
let result = xs.filter(fn(x: int): x > 1).map(fn(x: int): x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

reduce

アキュムレータ関数を使ってリストを単一の値に畳み込みます。最初の要素を初期値として使用します。

```
let xs = [1, 2, 3, 4, 5]
let total = reduce(xs, fn(a: int, b: int): a + b)
print(total)    # 15
```

fold

明示的な初期値とアキュムレータ関数を使ってリストを単一の値に畳み込みます。

```
let xs = [1, 2, 3, 4, 5]
let total = fold(xs, 0, fn(a: int, b: int): a + b)
print(total)    # 15
```

any

述語を満たす要素が 1 つ以上あれば true を返します。

```
let xs = [1, 2, 3, 4, 5]
print(any(xs, fn(x: int): x > 4))    # true
print(any(xs, fn(x: int): x > 9))    # false
```

all

すべての要素が述語を満たす場合に true を返します。

```
let xs = [2, 4, 6]
print(all(xs, fn(x: int): x > 0))    # true
print(all(xs, fn(x: int): x > 3))    # false
```

sum

全要素の合計を返します。

```
let xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

min

最小の要素を返します。

```
let xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

max

最大の要素を返します。

```
let xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

first

最初の要素を返します。空のリストに対して呼び出すとランタイムエラーになります。

```
let xs = [10, 20, 30]
print(first(xs))   # 10
```

last

最後の要素を返します。空のリストに対して呼び出すとランタイムエラーになります。

```
let xs = [10, 20, 30]
print(last(xs))    # 30
```

is_empty

リストが空であれば true を返します。

```
let xs = [1, 2, 3]
print(is_empty(xs)) # false
```

enumerate

(インデックス, 要素) のタプルのリストを返します。

```
let xs = [10, 20, 30]
let pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# for ループでのタプル分解
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

zip

2つのリストを (要素 1, 要素 2) のタプルのリストに結合します。結果の長さは短い方のリストと同じになります。

```
let xs = [1, 2, 3]
let ys = ["a", "b", "c"]
let pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# for ループでのタプル分解
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

insert

指定したインデックスに要素を挿入します。そのインデックス以降の要素は右にシフトされます。

```
var xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]
```

remove_at

指定したインデックスの要素を削除して返します。そのインデックス以降の要素は左にシフトされます。

```
var xs = [1, 2, 3, 4]
let v = remove_at(xs, 1)
print(v)    # 2
print(xs)   # [1, 3, 4]
```

remove

リストから指定した値の最初の出現を削除します。値が見つからない場合は何もしません。破壊的操作です。

```
var xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)    # [1, 3, 2, 4]
```

distinct

重複を排除した新しいリストを返します。元の順序は保持されます（最初の出現を残します）。元のリストは変更されません。

```
let xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))    # [1, 2, 3, 4]
print(xs)              # [1, 2, 3, 2, 1, 4] (変更なし)
```

flatten

ネストされたリスト（リストのリスト）を1段階フラット化します。新しいリストを返します。元のリストは変更されません。

```
let xs = [[1, 2], [3, 4]]
print(flatten(xs))    # [1, 2, 3, 4]
print(xs)             # [[1, 2], [3, 4]] (変更なし)
```

操作の計算量

操作	計算量
xs[i] インデックスアクセス	O(1)
append / append!	O(1) (均償)
pop	O(1)
first, last	O(1)
insert, remove_at	O(n)
sort / sort!	O(n log n)
filter, map, reduce, fold	O(n)
reverse / reverse!	O(n)
distinct	O(n)
len	O(1)

制約とエラー

制約	詳細
全要素は同一型	異なる型が混在するとコンパイルエラー
空リスト []	型推論できないためコンパイルエラー
範囲外アクセス	ランタイムエラー (exit(1))

マップ

概要

キーと値の対応表。ヒープ上に確保されます。

構文

```
let m = {"a": 1, "b": 2}
let m: Map<str, int> = {"a": 1, "b": 2}
```

キーアクセス

```
let m = {"a": 1, "b": 2}
print(m["a"])    # 1
```

挿入・更新

```
let m = {"a": 1}
m["b"] = 2      # 新規追加
m["a"] = 99     # 更新
```

len

```
let m = {"a": 1, "b": 2, "c": 3}
print(len(m))   # 3
```

print

```
let m = {"a": 1, "b": 2}
print(m)        # {a: 1, b: 2}
```

has_key

```
let m = {"a": 1, "b": 2}
print(m.has_key("a"))  # true
print(m.has_key("z"))  # false
```

keys

マップの全キーのリストを返します。

```
let m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
```

values

マップの全値のリストを返します。

```
let m = {"a": 1, "b": 2, "c": 3}
print(values(m))  # [1, 2, 3]
```

items

マップの全エントリの (キー, 値) タブルのリストを返します。

```
let m = {"a": 1, "b": 2}
let pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (マップ)

指定したキーのエントリをマップから削除します。キーが存在しない場合は何もしません。

```
let m = {"a": 1, "b": 2}
remove(m, "a")
print(m)    # {b: 2}
```

get

指定したキーの値を返します。キーが存在しない場合はデフォルト値を返します。

```
let m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

merge

2つのマップを結合した新しいマップを返します。キーが重複する場合は第2マップの値が優先されます。元のマップは変更されません。

```
let m1 = {"a": 1, "b": 2}
let m2 = {"b": 99, "c": 3}
let m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3
```

制約とエラー

制約	詳細
全キーは同一型	異なる型のキーが混在するとコンパイルエラー
全値は同一型	異なる型の値が混在するとコンパイルエラー
空マップ	型注釈が必要 (let m: Map<str, int> = {"a": 1} など)
存在しないキーアクセス	ランタイムエラー (exit(1))
キー検索	ハッシュテーブル (平均 O(1))
容量超過時	自動で2倍に拡張

セット

概要

同一型の要素を重複なしで保持するコレクション。ヒープ上に確保されます。

構文

```
let s = {1, 2, 3}
let s: Set<int> = {1, 2, 3}
```

対応する要素型

int, float, bool, str

in 演算子 (所属チェック)

```
let s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false
```

len

```
let s = {1, 2, 3}
print(len(s))    # 3
```

print

```
let s = {1, 2, 3}
print(s)         # {1, 2, 3}
```

add (要素追加)

重複する要素を追加した場合は無視されます。

```
let s = {1, 2, 3}
s.add(4)         # 追加
s.add(1)         # 既に存在するため無視
print(len(s))    # 4
```

remove (要素削除)

```
let s = {1, 2, 3}
s.remove(2)
print(2 in s)    # false
```

for 走査

```
let s = {10, 20, 30}
for x in s:
    print(x)
```

空セット

空セットは型注釈が必要です。

```
let s: Set<int> = {}
```

関数引数

```
fn has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

両方のセットの全要素を含む新しいセットを返します。

```
let a = {1, 2, 3}
let b = {3, 4, 5}
print(union(a, b))    # {1, 2, 3, 4, 5}
```


intersection

両方のセットに存在する要素のみを含む新しいセットを返します。

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(intersection(a, b))    # {2, 3}
```

difference

最初のセットにはあるが、2 番目のセットにはない要素を含む新しいセットを返します。

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(difference(a, b))    # {1}
```

symmetric_difference

いずれかのセットにあるが、両方にはない要素を含む新しいセットを返します。

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(symmetric_difference(a, b))    # {1, 4}
```

is_subset

最初のセットの全要素が 2 番目のセットに含まれている場合に true を返します。

```
let a = {1, 2}
let b = {1, 2, 3}
print(is_subset(a, b))    # true
print(is_subset(b, a))    # false
```

is_superset

最初のセットが 2 番目のセットの全要素を含んでいる場合に true を返します。

```
let a = {1, 2, 3}
let b = {1, 2}
print(is_superset(a, b))    # true
print(is_superset(b, a))    # false
```

制約とエラー

制約	詳細
全要素は同一型 空セット {}	異なる型が混在するとコンパイルエラー 型注釈が必要
要素検索	ハッシュテーブル (平均 $O(1)$)
容量超過時	自動で 2 倍に拡張

[English](#) | [日本語](#) | [繁體中文](#)

契約による設計 (Design by Contract)

Ry は Eiffel スタイルの契約による設計をサポートしています。事前条件 (require)、事後条件 (ensure)、構造体の不変条件 (invariant) を使用できます。契約違反時はプロセスが `exit(1)` で終了します。

事前条件 (require)

事前条件は関数の入口でチェックされます。関数が正しく呼び出されるために満たすべき条件を指定します。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  var new_balance: int = balance + amount
  return new_balance
```

事前条件が満たされない場合、以下のメッセージとともにプログラムが終了します：

```
Contract violation: require failed in deposit()
```

事後条件 (ensure)

事後条件はすべての return の直前にチェックされます。関数の戻り値について保証する条件を指定します。

result キーワード

ensure ブロック内では、result は戻り値を参照します。

```
fn abs(x: int) -> int:
  ensure:
    result >= 0
  if x < 0:
    return -x
  return x
```

old() 式

old(expr) は関数本体の実行前の式の値をキャプチャします。変更前後の状態を比較するのに便利です。

```
fn increment(x: int) -> int:
  ensure:
    result == old(x) + 1
  return x + 1
```

組み合わせ例

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure:
    result >= 0
    result == old(balance) + amount
  var new_balance: int = balance + amount
  return new_balance
```

構造体の不変条件 (invariant)

不変条件は構造体のインスタンスが常に満たすべき条件です。以下のタイミングでチェックされます：- 構築時
- フィールド代入後

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance

let a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1                # Contract violation: invariant failed
```

ルール

- `require` と `ensure` ブロックはオプションで、関数本体の前に記述します。
- 両方を使う場合、`require` は `ensure` の前に記述する必要があります。
- `result` と `old()` は `ensure` ブロック内でのみ使用できます。
- `invariant` は `record` 定義の末尾、全フィールド宣言の後に記述します。
- すべての契約違反は `exit(1)` でプログラムを終了し、診断メッセージを出力します。

[English](#) | [日本語](#) | [繁體中文](#)

制御構文リファレンス

if / elif / else

構文

```
if 条件式:
    # then ブロック
elif 条件式:
    # elif ブロック (複数可)
else:
    # else ブロック (省略可)
```

条件式の型

型	false になる値	true になる値
bool	false	true
int	0	非 0

`float` や `str` は条件式に直接使用できない。

例

```
let x = 10

if x > 5:
    print("big")
elif x == 5:
    print("five")
```

```
else:
    print("small")
```

スコープルール

- if / elif / else の各ブロックはそれぞれ独立したブロックスコープを持つ。
- ブロック内で宣言した変数はブロック外からアクセスできない。

```
if true:
    let y = 42
# y はここではアクセス不可
```

while

構文

```
while 条件式:
    # ループ本体
```

条件式が true の間、ループ本体を繰り返す。

例

```
let i = 0
while i < 5:
    print(i)
    i += 1
```

break / continue との組み合わせ

```
let i = 0
while true:
    if i >= 3:
        break
    i += 1
```

for

構文

```
# リスト / セット走査
for x in iterable_expr:
    # x に各要素が代入される

# range (0 始まり)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (開始・終了指定)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (ステップ指定)
```

```
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...
```

マップのキー・値走査

```
for k, v in map_expr:
    # k はキー、v は各エントリの値
```

タプル分解代入

2要素タプルのリスト（`enumerate()` や `zip()` の戻り値など）を走査する際、2つの変数に分解できます。_で値を破棄できます。

```
let xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)          # 11, 22

for _, x in enumerate(xs):
    print(x)              # インデックスを破棄
```

範囲演算子 (..)

.. 演算子は両端を含む整数の範囲を生成します。1 .. 5 は [1, 2, 3, 4, 5] を生成します。

```
for i in 1 .. 5:
    print(i)              # 1 2 3 4 5
```

例

```
let xs = [10, 20, 30]
for x in xs:
    print(x)

let s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)              # 0 1 2 3 4

for i in range(2, 6):
    print(i)              # 2 3 4 5

for i in range(0, 10, 2):
    print(i)              # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)              # 10 7 4 1

# マップの走査
let m = {"a": 1, "b": 2}
for k, v in m:
```

```

    print(k)
    print(v)

# 範囲演算子
for i in 1 .. 3:
    print(i)      # 1 2 3

```

break

- 最も内側のループ（while または for）を即座に脱出する。
- ループの外で使用するとコンパイルエラー。

```

for i in range(10):
    if i == 5:
        break      # i == 5 で脱出
    print(i)       # 0 1 2 3 4

```

エラー例

```

# ループ外での break はコンパイルエラー
break      # Error: break outside loop

```

continue

- 最も内側のループの現在のイテレーションを終了し、次のイテレーションへスキップする。
- ループの外で使用するとコンパイルエラー。

```

for i in range(5):
    if i == 2:
        continue   # i == 2 をスキップ
    print(i)       # 0 1 3 4

```

... (Ellipsis)

- 何もしない文（no-op）。空ブロックのプレースホルダーとして使用する。
- 関数ボディ、if/elif/else、while、for、match case など任意のブロック内で使用可能。

```

fn not_yet():
    ...

if true:
    ...
else:
    ...

```

match

構文

```

match 式:
    case パターン:

```

```

# 本体
case パターン if ガード条件:
# ガード付き本体
case _:
# ワイルドカード (何にでもマッチ)

```

パターンの種類

パターン	例	説明
ワイルドカード	<code>_</code>	何にでもマッチ
リテラル	<code>0, "hello", true</code>	値の等値比較
変数束縛	<code>n</code>	何にでもマッチし、変数に束縛
enum バリエーション	<code>Color::Red</code>	enum タグの比較 (単純な enum)
ADT enum バリエーション	<code>Shape::Circle(r)</code>	関連データを持つ enum バリエーションにマッチし、束縛する
<code>Some(x)</code>	<code>Some(v)</code>	<code>Option</code> が値ありの場合、中身を束縛
<code>None</code>	<code>None</code>	<code>Option</code> が値なしの場合
OR パターン	<code>1 \ 2 \ 3</code>	いずれかにマッチ

guard 節

`case` パターン `if` 条件式: の形式でガード条件を指定できる。パターンがマッチし、かつガード条件が真の場合にのみアームが実行される。

OR パターン

複数のパターンを `|` で結合し、いずれかにマッチさせることができます。変数束縛 (`n`、`Some(x)`、`Ok(v)`、`Err(e)`) は OR パターン内では使用できません。

```

match x:
  case 1 | 2 | 3:
    print("small")
  case _:
    print("other")

# enum の OR パターン
match color:
  case Color::Red | Color::Blue:
    print("warm or cool")
  case Color::Green:
    print("green")

```

網羅性チェック

- enum 型: すべてのバリエーションをカバーするか `_` が必要。OR パターンの各選択肢は個別にカウントされる。
- Option 型: `Some` と `None` の両方をカバーするか `_` が必要。
- bool 型: `true` と `false` の両方をカバーするか `_` が必要。
- int / float / str リテラル: `_` が必須。
- ガード付きアームは網羅性にカウントされない。

例

```
# enum マッチ
enum Color:
    Red
    Green
    Blue

match color:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")

# Option マッチ
let x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")

# リテラルマッチ
match x:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")

# guard 節
match x:
    case n if n > 0:
        print("positive")
    case n if n < 0:
        print("negative")
    case _:
        print("zero")
```

ADT enum マッチ

enum バリエントが関連データを持つ場合、バインディングパターンを使って値を取り出します。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

let s = Shape::Circle(3.14)
match s:
    case Shape::Circle(r):
        print(r)          # 3.14
```



```

case Shape::Rectangle(w, h):
    print(w)
    print(h)
case Shape::Point:
    print("point")

```

複数フィールドを持つバリエーションは、宣言順に各フィールドを別々の名前に束縛します。

スコープルール

- 各 case アームはブロックスコープを持つ。
- 変数束縛パターン (n) や Some(x) で束縛された変数はそのアーム内でのみ有効。

スコープルール

ブロックスコープ

- if / elif / else / while / for / match の各ブロックはブロックスコープを持つ。
- ブロック内で宣言した変数はブロックの終了と同時にスコープから外れる。

```

for i in range(3):
    let tmp = i * 2
# tmp はここではアクセス不可

```

```

if true:
    let a = 1
# a はここではアクセス不可

```

シャドーイング

- 内側のスコープで外側と同名の変数を宣言すると、内側のスコープ内では内側の変数が参照される。
- 内側スコープを抜けると外側の変数に戻る。

```

let x = 10
if true:
    let x = 99 # 外側の x をシャドーイング
    print(x)   # 99
print(x)      # 10

```

[English](#) | [日本語](#) | [繁體中文](#)

ディレクティブ

ディレクティブは宣言に付与できるコンパイル時メタデータです。@name 構文を使用します。

構文

```

@name
@name(key=value, ...)

```

ディレクティブは対象の宣言の前に配置します。複数のディレクティブを重ねることもできます。

対象

ディレクティブは以下の宣言に適用できます:

- fn - 関数定義

- record - 構造体定義
- let / var - 変数宣言
- record 定義内のフィールド

組み込みディレクティブ

@deprecated

宣言を非推奨としてマークします。非推奨のエンティティが使用（呼び出し、参照、アクセス）されると、コンパイル時に警告が出力されます。

関数に対して:

```
@deprecated
fn old_function() -> int:
    return 42
```

```
print(old_function())    # warning: 'old_function' is deprecated
```

型に対して:

```
@deprecated
record OldPoint:
    x: int
    y: int
```

```
let p = OldPoint(1, 2)  # warning: 'OldPoint' is deprecated
```

変数に対して:

```
@deprecated
let old_value = 99
```

```
print(old_value)        # warning: 'old_value' is deprecated
```

フィールドに対して:

```
record Config:
    @deprecated
    old_setting: int
    new_setting: int
```

```
let c = Config(1, 2)
print(c.old_setting)    # warning: 'Config.old_setting' is deprecated
print(c.new_setting)    # 警告なし
```

@native

ランタイム（組み込み）によって実装が提供される関数を宣言します。関数本体を持つことはできません。

基本構文:

```
@native
fn contains(s: str, sub: str) -> bool
```

```
print(contains("hello world", "world"))  # true
```

演算子オーバーロード:

```
@native
fn operator+(a: str, b: str) -> str
```

```
print("hello" + " world") # hello world
```

UFCS との組み合わせ:

```
@native
fn to_upper(s: str) -> str
```

```
print("hello".to_upper()) # HELLO
```

引数数の検証:

@native 宣言に型シグネチャが含まれている場合、コンパイラは呼び出し時に引数の数を検証します。オーバーロードされた関数（例: 1, 2, 3 引数の `range`）もサポートされ、いずれかのオーバーロードにマッチすれば検証を通過します。

標準ライブラリ宣言 (`core/`):

`core/` ディレクトリにはすべての組み込み関数の @native 宣言がカテゴリ別に格納されています:

ファイル	内容
<code>core/builtins.ry</code> <code>core/str.ry</code>	<code>print</code> , <code>len</code> , <code>range</code> , <code>enumerate</code> , <code>zip</code> , <code>exit</code> , <code>args</code> <code>contains</code> , <code>starts_with</code> , <code>ends_with</code> , <code>find</code> , <code>substring</code> , <code>char_at</code> , <code>replace</code> , <code>to_upper</code> , <code>to_lower</code> , <code>trim</code> , <code>trim_start</code> , <code>trim_end</code> , <code>repeat</code> , <code>reverse</code> , <code>split</code> , <code>join</code>
<code>core/convert.ry</code> <code>core/list.ry</code>	<code>to_int</code> , <code>to_float</code> , <code>to_str</code> <code>append</code> , <code>pop</code> , <code>insert</code> , <code>remove_at</code> , <code>slice</code> , <code>distinct</code> , <code>flatten</code> , <code>sort</code> , <code>first</code> , <code>last</code> , <code>is_empty</code>
<code>core/map.ry</code> <code>core/set.ry</code>	<code>keys</code> , <code>values</code> , <code>items</code> , <code>has_key</code> , <code>get</code> , <code>merge</code> <code>add</code> , <code>remove</code> , <code>union</code> , <code>intersection</code> , <code>difference</code> , <code>symmetric_difference</code> , <code>is_subset</code> , <code>is_superset</code>
<code>core/higher_order.ry</code>	<code>filter</code> , <code>map</code> , <code>reduce</code> , <code>fold</code> , <code>any</code> , <code>all</code> , <code>sum</code> , <code>min</code> , <code>max</code>

これらのファイルは `ry` 実行バイナリの近くに `core/` ディレクトリが存在する場合、プレリユードとして自動的にロードされます。プレリユードにより、組み込み関数呼び出し時の引数数検証が有効になります。

制約事項: - @native 関数は本体を持てません（シグネチャの後に `:` を付けるとエラー）。- 本体を付けるとパースエラー: @native function must not have a body. - 宣言した関数は既存の組み込み関数に対応している必要があります。対応していない場合はコンパイル時にエラーになります。

将来の拡張方向: - @native("libfoo.so") —外部共有ライブラリへの FFI バインディング。

パラメータ（将来拡張）

ディレクティブは将来の拡張に備え、パラメータ構文をサポートしています:

```
@deprecated(reason="use new_api instead")
fn old_api() -> int:
    return 0
```

現時点では、パラメータはパースされますが @deprecated ディレクティブでは使用されません。

注意事項

- 非推奨のエンティティは正常に動作します。警告が出力されるだけです。
- 警告は使用箇所でも出力され、定義箇所では出力されません。
- 非推奨のエンティティを定義しても、使用しなければ警告は出力されません。

- 未知のディレクティブ名はパースエラーになります。
- サポートされない対象 (if、while 等) にディレクティブを付与するとパースエラーになります。

[English](#) | [日本語](#) | [繁體中文](#)

エラーリファレンス

エラーフォーマット

ry はコンパイルエラーを Rust 風のリッチフォーマットで表示します。エラーの正確な位置がソースコードのコンテキストとともに表示されます:

```
error: cannot reassign let variable: x
  --> main.ry:5:1
  |
5 | x = 10
  | ^ cannot reassign let variable: x
```

各エラーメッセージには以下が含まれます: - エラーレベルとメッセージ (error: ...) - ファイル位置 (--> ファイル: 行: 列) - ソース行 (該当するコード) - キャレットインジケーター (^) 正確な列を指し示す

コンパイルエラー一覧

エラー内容	原因	例
未宣言変数への代入	宣言されていない変数に代入した	x = 1 (x が未宣言)
let 変数への再代入	let で宣言した変数に再代入した	let x = 1 → x = 2
同名変数の再宣言	同スコープで同名の変数を再宣言した	let x = 1 → let x = 2
変数の型変更再代入	変数に異なる型の値を代入した	let x = 1 → x = 3.14
型アノテーション不一致	宣言時の型と代入値の型が異なる	let x: int = 3.14
オーバーロード戻り値型の重複	引数型が同一で戻り値型のみ異なるオーバーロードを定義した	引数 (int, int) で戻り値が int と float の2つの関数
オーバーロード解決失敗	引数型に一致するオーバーロードが存在しない	fn add(a: int, b: int) に add(1.0, 2.0)
ビット演算に float を渡す	&, , ^, ~, <<, >> に float 型を渡した	3.14 & 1
空リスト	空リストリテラル [] から型を推論できない	let xs = []
空マップ	空マップリテラル {} から型を推論できない	let m = {}
タプル範囲外インデックス	タプルの存在しないインデックスにアクセスした	let t = (1, 2) → t.2
ループ外での break/continue	for/while ループの外で break または continue を使用した	関数トップレベルで break
ブロック内でのモジュールインポート	from 文を関数・条件ブロック内で使用した	関数内で from math

エラー内容	原因	例
循環インポート	モジュールが相互にインポートし合っている	<code>a.ry</code> が <code>b.ry</code> を、 <code>b.ry</code> が <code>a.ry</code> をインポート
同一フィールド名の重複	構造体内で同じフィールド名を2回定義した	<code>type T: x: int</code> の中に <code>x</code> を2つ定義
<code>match</code> の網羅性不足	<code>match</code> がすべてのパターンを網羅していない	<code>enum</code> の一部バリエーションが未カバー、 <code>Option</code> で <code>None</code> が欠落、リテラルに <code>_</code> なし
!! 戻り値型の不一致	(<code>T, Error?</code>) を返さない関数内で <code>!!</code> を使用した	<code>int</code> を返す関数内で <code>!!</code> を使用
!! オペランド型の不一致	(<code>T, Error?</code>) でない値に <code>!!</code> を適用した	通常の <code>int</code> に <code>!!</code> を適用

コンパイルエラーの例

```
# let 変数への再代入
let x = 10
x = 20    # エラー

# 型変更再代入
let n = 1
n = "hello"    # エラー: int型に str型を代入

# 空リスト
let xs = []    # エラー: 型推論不可

# ループ外での break
break    # エラー: ループの外

# ブロック内でのインポート
fn foo():
    from math    # エラー: トップレベルのみ

# 同一フィールド名の重複
record Bad:
    x: int
    x: float    # エラー: xが重複
```

ランタイムエラー一覧

エラー内容	原因	例
リスト範囲外アクセス	リストのインデックスが範囲を超えている	<code>let xs = [1, 2, 3]</code> → <code>xs[5]</code>
マップ存在しないキーアクセス	マップに存在しないキーを参照した	<code>let m = {"a": 1}</code> → <code>m["b"]</code>
契約違反	<code>require · ensure · invariant</code> の条件が <code>false</code> と評価された	契約による設計 を参照

すべてのランタイムエラーはプロセスを `exit(1)` で終了します。

ランタイムエラーの例

```
# リスト範囲外アクセス
let xs = [1, 2, 3]
print(xs[10])    # ランタイムエラー: exit(1)
```

```
# マップ存在しないキーアクセス
let m = {"a": 1}
print(m["z"])    # ランタイムエラー: exit(1)
```

[English](#) | [日本語](#) | [繁體中文](#)

関数リファレンス

関数定義の構文

fn 関数名 (引数名: 型, ...) -> 戻り値型:

```
    # 本体
    return 値
```

- 引数の型は必須。
- 戻り値型は省略可能（省略時は `Unit`）。
- 本体はインデントされたブロック。
- 関数には `require`（事前条件）と `ensure`（事後条件）を定義できます。[契約による設計](#)を参照。

命名規則: 関数名と引数名は `snake_case`（例: `add`, `get_value`, `map_list`）を使用する必要があります。コンパイラがこの規則を強制します。

```
fn add(a: int, b: int) -> int:
    return a + b
```

```
fn greet(name: str):
    print("Hello, " + name)    # 戻り値型は Unit
```

引数と戻り値の型

項目	説明
引数型	必須。すべての引数に型アノテーションが必要
戻り値型	省略可能。省略時は <code>Unit</code> （void 相当）
<code>Unit</code>	値を返さない関数の戻り値型

```
fn no_return(x: int):    # 戻り値型 Unit（省略）
    print(x)
```

```
fn get_value() -> int:    # 戻り値型 int
    return 42
```

再帰

関数は自身を呼び出せる。

```
fn factorial(n: int) -> int:
    if n <= 1:
```

```
    return 1
return n * factorial(n - 1)
```

オーバーロード

引数の数や型が異なる同名の関数を複数定義できる。

ルール

- 引数の数または型が異なれば同名の関数を定義可能。
- 呼び出し時は引数の型と数に基づいて適切な関数が選択される。
- 戻り値型だけが異なるオーバーロードはできない。

```
fn area(side: int) -> int:
    return side * side
```

```
fn area(w: int, h: int) -> int:
    return w * h
```

```
let a = area(5)      # 25
let b = area(3, 4)   # 12
```

Unit 型関数

戻り値のない関数は Unit を返す。戻り値型は省略可能。

```
fn log(msg: str):
    print(msg)
```

```
fn log_typed(msg: str) -> Unit:
    print(msg)
```

ラムダ関数

無名関数をその場で定義できる。

構文

単一式（式の値が返る。戻り値型は推論）

```
fn(引数名: 型, ...): 式
```

複数行ブロック

```
fn(引数名: 型, ...):
```

```
    # 複数の文
    return 値
```

戻り値型の明示（省略可能）

```
fn(引数名: 型, ...) -> 戻り値型: 式
```

例

```
let double = fn(x: int): x * 2
let result = double(5)    # 10

let add = fn(a: int, b: int): a + b
let sum = add(3, 4)      # 7

# 複数行ラムダ
let abs = fn(x: int):
  if x < 0:
    return -x
  return x
```

クロージャ

ラムダ関数は定義された時点の外側スコープの変数を値でキャプチャする。

```
let base = 10
let add_base = fn(x: int): x + base    # base を値でキャプチャ

base = 99    # キャプチャ済みの値には影響しない
let r = add_base(5)    # 15 (キャプチャ時の base = 10 を使用)
```

キャプチャルール

項目	内容
キャプチャ方式	値キャプチャ (コピー)
キャプチャタイミング	ラムダ定義時
外側変数の変更の影響	ない (コピーのため)

関数型

関数を値として扱うための型。

構文

fn(引数型 1, 引数型 2, ...) -> 戻り値型

例

```
let f: fn(int) -> int = fn(x: int): x * 2
let g: fn(int, int) -> int = fn(a: int, b: int): a + b

fn apply(func: fn(int) -> int, x: int) -> int:
  return func(x)

let result = apply(f, 5)    # 10
```

高階関数

関数を引数として受け取ったり、戻り値として返したりできる。

```
fn map_list(xs: List<int>, f: fn(int) -> int) -> List<int>:
    let result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

let doubled = map_list([1, 2, 3], fn(x: int): x * 2)
# [2, 4, 6]
```

UFCS（統一関数呼び出し構文）

a.f(b) の形式で f(a, b) を呼び出せる。メソッドチェーンに使いやすい。

構文

```
# 通常呼び出し
f(a, b)

# UFCS 呼び出し（等価）
a.f(b)
```

チェーン

```
fn double(x: int) -> int:
    return x * 2

fn add_one(x: int) -> int:
    return x + 1

let result = 5.double().add_one() # double(5) → 10, add_one(10) → 11
```

フィールドアクセスとの混在

フィールドアクセス (.field) と UFCS (.method()) は同じドット記法で書けるが、引数の有無で区別される。

```
let p = Point(3, 4)
let len = p.x.to_float() # フィールドアクセス + UFCS
```

演算子オーバーロード

ユーザー定義型に対して演算子の振る舞いを定義できる。

構文

```
# 二項演算子（引数 2 個）
fn operator<op>(a: 型, b: 型) -> 戻り値型:
    ...

# 単項演算子（引数 1 個）
```

```
fn operator<op>(a: 型) -> 戻り値型:
    ...
```

オーバーロード可能な演算子

種別	演算子
算術（二項）	+ - * / % ** //
比較（二項）	== != < <= > >=
ビット（二項）	& \ ^ << >>
論理（二項）	and or
単項	- ~ not

二項 / 単項の区別

引数の個数で区別する。

```
record Vec2:
    x: float
    y: float

# 二項 +
fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

# 単項 -
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)

# 比較
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

let v1 = Vec2(1.0, 2.0)
let v2 = Vec2(3.0, 4.0)
let v3 = v1 + v2      # Vec2(4.0, 6.0)
let v4 = -v1          # Vec2(-1.0, -2.0)
```

[English](#) | [日本語](#) | [繁體中文](#)

演算子リファレンス

優先順位テーブル

優先順位は数字が小さいほど高い（先に評価される）。

優先順位	演算子	説明	結合性
0	!!	エラー伝播（後置）	左
1	()	グループ化	—
2	+x -x ~x	単項正・負、ビット NOT	右
3	**	累乗	右結合
3.5	as	型キャスト	左
4	* / % //	乗算・除算・剰余・整数除算	左
5	+ -	加算・減算	左

優先順位	演算子	説明	結合性
6	<< >> >>>	ビットシフト	左
7	&	ビット AND	左
8	^	ビット XOR	左
9	\	ビット OR	左
10	== != < <= > >= in not in	比較・所属	左
11	not	論理 NOT	右
12	and	論理 AND	左
13	or	論理 OR	左
13.5	??	null 合体	左
14	?:	三項条件	右

算術演算子

演算子	説明	例
+	加算 / 文字列結合	1 + 2 → 3、"a" + "b" → "ab"
-	減算	5 - 3 → 2
*	乗算 / 文字列繰り返し	4 * 3 → 12、"ab" * 3 → "ababab"
/	除算（常に float）	7 / 2 → 3.5
//	整数除算（切り捨て）	7 // 2 → 3
%	剰余	7 % 3 → 1
**	累乗（常に float）	2 ** 10 → 1024.0
-x	単項マイナス	-5, -3.14
+x	単項プラス	+5（符号変更なし）

```
let a = 10 // 3      # 3 (int)
let b = 10 / 3       # 3.3333... (float)
let c = 2 ** 8       # 256.0 (float)
let s = "foo" + "bar" # "foobar"
```

比較演算子

すべて bool を返す。

演算子	説明
==	等しい
!=	等しくない
<	より小さい
<=	以下
>	より大きい
>=	以上

- 数値型 (int / float) と bool に対して使用可能。
- str 同士は辞書順（バイト順）で比較。
- in 演算子はセット、リスト、マップに対する所属チェックに使用 (x in s)。
- not in 演算子は in の否定 (x not in s)。
- マップの場合、in はキーの存在を確認します。

```
let x = 3 < 5      # true
let y = "abc" < "abd" # true (辞書順)
```

```

let s = {1, 2, 3}
let z = 2 in s      # true
let w = 4 not in s  # true
let xs = [1, 2, 3]
let a = 2 in xs     # true (リスト線形探索)
let m = {"a": 1}
let b = "a" in m    # true (マップキー検索)

```

論理演算子

演算子	説明	型
and	論理 AND	bool × bool → bool
or	論理 OR	bool × bool → bool
not	論理 NOT	bool → bool

```

let a = true and false  # false
let b = true or false   # true
let c = not true        # false

```

ビット演算子

int 型のみ使用可能。float や bool に適用するとコンパイルエラー。

演算子	説明	例
&	ビット AND	0b1100 & 0b1010 → 0b1000
\	ビット OR	0b1100 \ 0b1010 → 0b1110
^	ビット XOR	0b1100 ^ 0b1010 → 0b0110
~	ビット NOT (単項)	~0 → -1
<<	左シフト	1 << 4 → 16
>>	算術右シフト	16 >> 2 → 4
>>>	論理右シフト	-1 >>> 1 → 9223372036854775807

```

let flags = 0b0001 \| 0b0010  # 3
let masked = flags & 0b0011    # 3
let shifted = 1 << 8           # 256

```

三項条件演算子

```
let x = condition ? true_value : false_value
```

condition を評価し、真であれば true_value を、偽であれば false_value を返します。両方の分岐は同じ型でなければなりません。右結合なので、ネストされた三項演算子は右から左に結合されます。

```

let x = 3 > 2 ? 10 : 20      # 10
let s = false ? "yes" : "no" # "no"

```

ネスト (右結合)

```
let y = true ? (false ? 1 : 2) : 3  # 2
```

範囲演算子

.. 演算子は両端を含む整数の範囲を生成します。

```
let xs = 1 .. 5    # [1, 2, 3, 4, 5]

for i in 1 .. 3:
    print(i)       # 1 2 3
```

結果は左オペランドから右オペランドまで（両端含む）のすべての整数を含む `List<int>` です。

null 合体演算子 (??)

```
let x = option_val ?? default_val
```

`option_val` が `Some(v)` の場合は `v` を返します。それ以外の場合は `default_val` を返します。右辺のオペランドは `Option` の内部型と同じ型でなければなりません。

```
let a: int? = Some(10)
let b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0
```

複合代入演算子

変数を更新するショートハンド。 `x op= y` は `x = x op y` と等価。

演算子	等価な式
<code>x += y</code>	<code>x = x + y</code>
<code>x -= y</code>	<code>x = x - y</code>
<code>x *= y</code>	<code>x = x * y</code>
<code>x /= y</code>	<code>x = x / y</code>
<code>x %= y</code>	<code>x = x % y</code>
<code>x /= y</code>	<code>x = x // y</code>
<code>x **= y</code>	<code>x = x ** y</code>
<code>x &= y</code>	<code>x = x & y</code>
<code>x \ = y</code>	<code>x = x \ y</code>
<code>x ^= y</code>	<code>x = x ^ y</code>
<code>x <<= y</code>	<code>x = x << y</code>
<code>x >>= y</code>	<code>x = x >> y</code>

```
var x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
```

演算の型規則

演算	左辺型	右辺型	結果型
<code>+ - *</code>	<code>int</code>	<code>int</code>	<code>int</code>
<code>+ - *</code>	<code>float</code>	<code>int / float</code>	<code>float</code>

演算	左辺型	右辺型	結果型
+ - *	int	float	float
/	任意の数値	任意の数値	float
//	任意の数値	任意の数値	int
**	任意の数値	任意の数値	float
%	int	int	int
%	float または int (片方 float)	—	float
+	str	str	str
== != < <= > >=	数値 / bool / str	同型	bool
*	str	int	str
in	任意	Set / List / Map<K, V>	bool
not in	任意	Set / List / Map<K, V>	bool
& \ ^ ~ << >> >>>	int	int	int
and or not	bool	bool	bool

演算子オーバーロード

ユーザー定義型に対して演算子の動作を定義できます。

構文

```
# 二項演算子 (引数 2 個)
fn operator+(a: MyType, b: MyType) -> MyType:
    ...

# 単項演算子 (引数 1 個)
fn operator-(a: MyType) -> MyType:
    ...
```

オーバーロード可能な演算子一覧

種別	演算子
算術 (二項)	+ - * / % ** //
比較 (二項)	== != < <= > >=
ビット (二項)	& \ ^ << >> >>>
論理 (二項)	and or
単項	- ~ not

二項 / 単項の区別

引数の個数で区別します。

```
# 二項 -
fn operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)

# 単項 -
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

[English](#) | [日本語](#) | [繁體中文](#)

パッケージリファレンス

概要

Ry はパッケージシステムでコードを管理します。パッケージは単一の .ry ファイル、またはディレクトリ（複数の .ry ファイルを含む）のいずれかです。from 文でパッケージをインポートします。

std パッケージ（標準ライブラリ）はすべてのプログラムに自動的にインポートされます。

インポート構文

全定義インポート

```
from math
```

パッケージ内のすべての関数・型をインポートします。

選択インポート

```
from math import add
```

指定した定義のみをインポートします。

複数選択インポート

```
from math import add, sub
```

カンマ区切りで複数の定義を選択インポートします。

パッケージ解決

ドット区切りのパッケージ名は以下のように解決されます:

インポート文	解決先
from math	math/ ディレクトリ（パッケージ）または math.ry ファイル
from utils.math	utils/math/ ディレクトリまたは utils/math.ry ファイル
from std.str	std/str/ ディレクトリまたは std/str.ry ファイル

解決順序

各検索パスに対して: 1. ディレクトリ ({path}/) — 存在すればディレクトリ内の全 .ry ファイルを読み込む (パッケージ) 2. ファイル ({path}.ry) — 単一ファイル (後方互換)

ディレクトリパッケージ

パッケージがディレクトリに解決された場合: - ディレクトリ内のすべての .ry ファイルが自動的に読み込まれる - _ で始まるファイルは除外される - 特別なエントリファイル (__init__.py のようなもの) は不要 - ディレクトリ内のファイルで定義されたすべての関数・型がエクスポートされる

```
mypackage/  
  math.ry      # fn add(), fn sub()  
  string.ry    # fn concat()
```

```
from mypackage          # add, sub, concat をインポート
from mypackage import add # add のみインポート
```

標準ライブラリ (std)

std パッケージはすべてのプログラムに自動的にインポートされます。提供する機能: - 組み込み関数 (print, len, range など) - 文字列関数 (contains, find, replace など) - 型変換関数 (to_int, to_float, to_str) - コレクション関数 (map, filter, sort など)

特定の定義を明示的にインポートすることもできます:

```
from std.str import contains
```

RY_HOME

標準ライブラリは \$RY_HOME/lib/std/ にインストールされます。RY_HOME のデフォルト値は ~/.ry です。

```
export RY_HOME="$HOME/.ry" # デフォルト
```

検索パスの優先順位

1. インポート元ファイルのディレクトリ
 2. \$RY_HOME/lib (標準ライブラリの場所)
 3. 実行ファイル相対の lib/ ディレクトリ
 4. RY_PATH 環境変数に含まれるパス (コロン区切り)
-

RY_PATH 環境変数

RY_PATH にコロン区切りでディレクトリを指定すると、パッケージ検索パスに追加されます。

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

制約

制約	詳細
使用可能な位置	トップレベルのみ (関数・ブロック内は不可)
二重インポート	自動でスキップ (エラーにならない)
循環インポート	コンパイルエラー

エラー例: ブロック内でのインポート

```
fn main():
    from math # エラー: トップレベル以外ではインポート不可
```

OK: 同じパッケージを複数回インポートしてもエラーにならない

```
from math
from math # スキップされる
```

パッケージファイルの作成

単一ファイルパッケージ

```
# math.ry
fn add(a: int, b: int) -> int:
    return a + b

fn sub(a: int, b: int) -> int:
    return a - b

# main.ry
from math import add, sub

print(add(1, 2))    # 3
print(sub(5, 3))    # 2
```

ディレクトリパッケージ

```
mylib/
  math.ry
  string.ry

# main.ry
from mylib import add, concat
```

[English](#) | [日本語](#) | [繁體中文](#)

プロジェクト管理

ry init - プロジェクト初期化

カレントディレクトリを Ry プロジェクトとして初期化します。

```
ry init
```

生成されるファイル・ディレクトリ

```
my-project/
  ry.toml          # プロジェクト設定ファイル
  src/
    main.ry        # エントリポイント（サンプルコード）
```

動作

1. ry.toml が既に存在する場合はエラー終了する
2. src/ ディレクトリを作成（存在しなければ）
3. ry.toml を生成（name はカレントディレクトリ名）
4. src/main.ry を生成（存在しなければスキップ）

ry self-update - セルフアップデート

ry 自身を最新バージョンに更新します。GitHub Releases からバイナリをダウンロードし、現在の実行ファイルを置換します。

```
ry self-update           # 最新安定版に更新
ry self-update --nightly # 最新 nightly プレリリースに更新
ry self-update v0.0.1    # 指定バージョンに更新
```

動作

1. 現在のバージョンを表示
2. 引数に応じて更新対象のバージョンを解決
 - 引数なし: GitHub の最新安定リリース (/releases/latest)
 - --nightly: 最新のプレリリース
 - バージョン指定: 指定されたタグのリリース
3. 現在のバージョンと同じ場合は "Already up to date." で終了
4. バイナリをダウンロードして現在の実行ファイルを置換

注意事項

- 実行には curl および tar コマンドが必要です
- パーミッション不足でバイナリの置換に失敗した場合、sudo の使用を促すメッセージが表示されます (sudo は自動的に実行されません)
- ダウンロードはテンポラリディレクトリに対して行われますが、ファイルシステムをまたぐ cp フォールバックが中断された場合、バイナリが不完全な状態になる可能性があります

ry.toml 設定ファイル

プロジェクトのメタデータとパス設定を TOML 形式で記述します。

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"
```

```
[paths]
src = "src"
```

[project] セクション

キー	説明
name	プロジェクト名 (初期化時はディレクトリ名)
version	バージョン文字列
entry	エントリポイントとなるソースファイル

[paths] セクション

キー	説明
src	ソースコードディレクトリ

TOML サブセット仕様

ry.toml は以下の TOML サブセットをサポートします。

- セクションヘッダ: [section]
- キー・値ペア: key = "value" (文字列値のみ)

- コメント: # から行末まで
- 空行は無視される

[English](#) | [日本語](#) | [繁體中文](#)

正規表現関数リファレンス

正規表現関数の一覧です。すべての関数は UFCS 記法をサポートしています。パターン文字列は標準的な正規表現構文を使用します。

注意: Phase 1 ではパターンは通常の文字列として渡します。専用の正規表現リテラル構文は将来追加される可能性があります。

関数一覧

関数	シグネチャ	説明
<code>regex_match</code>	<code>(str, str) -> bool</code>	テキスト全体がパターンにマッチするか
<code>regex_search</code>	<code>(str, str) -> int</code>	最初のマッチの開始位置を返す (見つからない場合 -1)
<code>regex_replace</code>	<code>(str, str, str) -> str</code>	マッチした部分を置換文字列で置換
<code>regex_split</code>	<code>(str, str) -> List<str></code>	パターンマッチで分割
<code>regex_find_all</code>	<code>(str, str) -> List<str></code>	重複しないすべてのマッチを返す

サポートするパターン構文

構文	説明	例
<code>abc</code>	リテラル文字	"hello"
<code>.</code>	任意の 1 文字 (改行を除く)	"a.c" は "abc", "aXc" にマッチ
<code> </code>	選択	"cat dog" は "cat" か "dog" にマッチ
<code>*</code>	0 回以上の繰り返し	"a*" は "", "a", "aaa" にマッチ
<code>+</code>	1 回以上の繰り返し	"a+" は "a", "aaa" にマッチ
<code>?</code>	0 回または 1 回	"a?" は "" か "a" にマッチ
<code>(...)</code>	グループ	"(ab)+" は "abab" にマッチ
<code>[abc]</code>	文字クラス	"[aeiou]" は母音にマッチ
<code>[a-z]</code>	文字範囲	"[a-z]+" は小文字の単語にマッチ
<code>[^...]</code>	否定文字クラス	"[^0-9]" は数字以外にマッチ
<code>^</code>	文字列先頭アンカー	"^hello"
<code>\$</code>	文字列末尾アンカー	"world\$"
<code>\d</code>	数字 [0-9]	"\d+" は数値にマッチ
<code>\D</code>	数字以外 [^0-9]	
<code>\w</code>	単語文字 [a-zA-Z0-9_]	"\w+" は識別子にマッチ
<code>\W</code>	単語文字以外	
<code>\s</code>	空白文字	"\s+" はスペースやタブにマッチ

構文	説明	例
<code>\s</code>	空白文字以外	
<code>\.</code>	エスケープされた特殊文字	<code>"\."</code> はリテラルの <code>.</code> にマッチ

使用例

`regex_match`

```
print(regex_match("[a-z]+", "hello")) # true
print(regex_match("[0-9]+", "hello")) # false
```

`regex_search`

```
let pos = regex_search("[0-9]+", "abc123def")
print(pos) # 3
```

`regex_replace`

```
let s = regex_replace("[0-9]+", "a1b2c3", "X")
print(s) # aXbXcX
```

`regex_split`

```
let parts = regex_split("\\s+", "hello world foo")
print(len(parts)) # 3
print(parts[0]) # hello
```

`regex_find_all`

```
let matches = regex_find_all("[0-9]+", "a1b23c456")
print(len(matches)) # 3
print(matches[0]) # 1
print(matches[1]) # 23
```

UFCS 記法

```
# pattern.function(text, ...)
let m = "[a-z]+".regex_match("hello")
print(m) # true
```

[English](#) | [日本語](#) | [繁體中文](#)

構造体リファレンス

概要

構造体はスタック上の値型です。`record` キーワードで定義します。構造体には `invariant` 節で不変条件を定義できます。[契約による設計](#) を参照。

命名規則: 構造体名は PascalCase (例: `Point`、`Rectangle`) を使用する必要があります。フィールド名は `snake_case` を使用します。コンパイラがこれらの規則を強制します。

定義構文

```
record 型名:  
    フィールド名: 型  
    フィールド名: 型
```

例

```
record Point:  
    x: int  
    y: int  
  
record Rectangle:  
    width: float  
    height: float
```

コンストラクタ

フィールド定義順に引数を渡します。名前付き引数はサポートされていません。

```
let p = Point(10, 20)  
let r = Rectangle(3.0, 4.5)
```

フィールドアクセス

ドット記法でフィールドを読み取ります。

```
let p = Point(10, 20)  
print(p.x)    # 10  
print(p.y)    # 20
```

フィールド代入

変数宣言	フィールド代入
var	可能
let	コンパイルエラー

```
var p = Point(10, 20)  
p.x = 100    # OK: var 変数  
  
let q = Point(10, 20)  
q.x = 100    # エラー: let 変数のフィールドは変更不可
```

関数引数・戻り値としての使用

```
fn distance(p: Point) -> float:  
    return (p.x * p.x + p.y * p.y) as float  
  
fn make_point(x: int, y: int) -> Point:  
    return Point(x, y)
```

ネスト構造体

構造体を別の構造体のフィールドとして使用できます。

```
record Point:
  x: int
  y: int

record Circle:
  center: Point
  radius: float

let c = Circle(Point(0, 0), 1.0)
print(c.center.x)  # 0
```

制約とエラー

制約	詳細
同一フィールド名の重複	コンパイルエラー
let 変数のフィールド代入	コンパイルエラー
print に構造体を直接渡す	コンパイルエラー (print 非対応)

エラー例: 同一フィールド名の重複

```
record Bad:
  x: int
  x: int  # エラー
```

エラー例: print に構造体を渡す

```
let p = Point(1, 2)
print(p)  # エラー
```

列挙型 (enum)

概要

列挙型は名前付き定数の集合です。内部的には i64 整数 (0, 1, 2, ...) として表現されます。

定義構文

```
enum 型名:
  バリエーション名
  バリエーション名
  ...
```

例

```
enum Color:
  Red
  Green
  Blue
```

バリエントアクセス

`::` 演算子でバリエントにアクセスします。

```
let c = Color::Red
print(c)    # Red
```

比較

enum 値は整数なので `==` / `!=` でそのまま比較できます。

```
print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true
```

if 文での使用

```
let c = Color::Green
if c == Color::Red:
    print("red")
elif c == Color::Green:
    print("green")
else:
    print("blue")
```

関数引数

型名として enum 名を使用します。

```
fn is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

print

`print()` でバリエント名が出力されます。

```
let c = Color::Blue
print(c)    # Blue
```

制約とエラー

制約	詳細
バリエントアクセスは <code>EnumName::VariantName</code>	<code>::</code> 演算子が必須
バリエント値は自動割り当て	0, 1, 2, ... の連番 (手動指定不可)
比較は整数比較	<code>==</code> , <code>!=</code> が使用可能

[English](#) | [日本語](#) | [繁體中文](#)

テスト機能

Ry は RSpec 風のテスト構文を内蔵しています。ry test サブコマンドでテストファイルを実行します。

実行方法

```
ry test          # プロジェクト内の *.test.ry を自動検出して実行
ry test test_file.ry # 特定のテストファイルを実行
```

終了コードは全テスト成功時に 0、失敗がある場合は 1 です。

自動検出モード

`ry test` を引数なしで実行すると:

1. `ry.toml` を探してプロジェクトルートを特定
2. プロジェクトルート以下の `*.test.ry` ファイルを再帰的に検出 (`.git`、`build`、`node_modules` はスキップ)
3. 各ファイルを実行し、結果を集計

構文

`describe / it`

```
describe("説明文"):
  it("テストケース名"):
    # テスト本体
    expect(実際の値).to_eq(期待値)
```

- `describe` と `it` は**トレイリングブロック構文**を使用: 関数呼び出しの後に `:` を付けるとインデントブロックがラムダとして最後の引数に渡される
- `describe` ブロック内には `it` ブロックやその他の文 (変数宣言など) を記述可能
- 各 `it` ブロックは独立したテストケース
- `describe / expect` は `ry test` でのみ使用可能 (通常の `ry` 実行ではコンパイルエラー)

トレイリングブロック構文

任意の関数呼び出しにトレイリングブロック構文が使えます。() の後に `:` を付けると、インデントブロックが引数なしラムダとして最後の引数に渡されます:

```
# 以下は等価:
foo("arg"):
  bar()
```

```
foo("arg", fn():
  bar()
)
```

`expect / マッチャー`

マッチャー	説明	対応型
<code>to_eq(expected)</code>	等値比較	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_not_eq(expected)</code>	等しくないこと	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>
<code>to_be_true()</code>	<code>true</code> であること	<code>bool</code>
<code>to_be_false()</code>	<code>false</code> であること	<code>bool</code>
<code>to_be_none()</code>	<code>None</code> であること	<code>Option</code>
<code>to_be_some()</code>	<code>Option</code> が <code>Some</code> であること	<code>Option</code>
<code>to_contain(val)</code>	コンテナが値を含むこと	<code>List</code> , <code>Set</code> , <code>str</code>

出力形式

Calculator

```
+ adds numbers
+ subtracts
- fails test (赤色)
  line 10: expected 3, got 2
```

2 passed, 1 failed

- + は成功（緑色）、- は失敗（赤色）
 - 失敗時は行番号と期待値/実際の値を表示
-

例

```
describe("Arithmetic"):
  it("adds integers"):
    expect(1 + 2).to_eq(3)

  it("compares strings"):
    expect("hello").to_eq("hello")

  it("checks booleans"):
    expect(3 > 1).to_be_true()

describe("Booleans"):
  it("false check"):
    expect(1 > 2).to_be_false()
```

モック

mock(fn_name, replacement)

現在の it ブロック内で関数をモック実装に差し替えます。it ブロック終了時にモックは自動的にクリアされます。

```
fn fetch_data() -> str:
  return "real data"

describe("mocking"):
  it("replaces function"):
    mock(fetch_data, fn(): "fake")
    expect(fetch_data()).to_eq("fake")

  it("auto-restores"):
    expect(fetch_data()).to_eq("real data")
```

- 第1引数は関数名（識別子、文字列ではない）
- 第2引数は差し替え用ラムダ
- 差し替え関数は元の関数と同じ引数型・戻り値型である必要がある
- it ブロック終了時にモックは自動復元される

`verify(fn_name)`

モック済み関数の呼び出し回数を `int` で返します。

```
describe("verify"):
  it("counts calls"):
    mock(fetch_data, fn(): "fake")
    fetch_data()
    fetch_data()
    expect(verify(fetch_data)).to_eq(2)
```

モックの制限事項

- オーバーロードされた関数のモックは非対応
- キャプチャ付きクロージャでのモックは非対応（プレーンラムダのみ）
- `@native fn` のモックは非対応

制限事項

- `describe` のネストは未対応
- `before_each` / `after_each` は未対応

[English](#) | [日本語](#) | [繁體中文](#)

型リファレンス

型一覧

型	内部表現	リテラル例	説明
<code>int</code>	<code>i64</code>	<code>42, -7, 0xFF, 0b1010</code> (専用リテラルなし)	64 ビット符号付き整数
<code>byte</code>	<code>i8</code>		符号なし 8 ビット整数 (0-255)。型アノテーション <code>let b: byte = 42</code> で使用
<code>float</code>	<code>f64</code>	<code>3.14, 0.5</code>	64 ビット浮動小数点数
<code>bool</code>	<code>il</code>	<code>true, false</code>	真偽値
<code>str</code>	<code>ptr</code>	<code>"hello", "", "a\nb"</code>	文字列（ヒープ上の不変バイト列）
<code>Unit</code>	<code>void</code>	(戻り値なし)	戻り値型省略時の暗黙の戻り値型
<code>Option<T></code>	<code>{ i1, T }</code>	<code>Some(42), None</code>	値が存在するかもしれない型
<code>(T1, T2, ...)</code>	LLVM StructType (literal)	<code>(1, 3.14)</code>	タプル型
<code>List<T></code>	<code>ptr</code> （ヒープ）	<code>[1, 2, 3]</code>	動的配列
<code>Map<K, V></code>	<code>ptr</code> （ヒープ）	<code>{"a": 1}</code>	ハッシュマップ
<code>Set<T></code>	<code>ptr</code> （ヒープ）	<code>{1, 2, 3}</code>	重複なしの集合
<code>fn(T1, T2) -> R</code> ユーザー定義型	<code>ptr</code> （関数ポインタ） LLVM StructType (named)	<code>fn(x: int): x * 2</code> <code>record Point: ...</code>	関数型 <code>record</code> キーワードで定義する構造体
<code>enum</code>	<code>i64</code> / タグ付きユニオン	<code>Color::Red,</code> <code>Shape::Circle(3.14)</code>	<code>enum</code> キーワードで定義する列挙型（関連データをサポート）

型	内部表現	リテラル例	説明
Error	{ ptr, i64 }	Error("msg"), Error("msg", 404)	組み込みエラー型
T1 \ T2	{ i64, [N x i8] }	int \ str	union 型（複数の型のいずれかを保持）
int リテラル	i64	42, 0 \ 1	int リテラル型（値の制約）
str リテラル	ptr	"N" \ "S"	str リテラル型（値の制約）
範囲	i64	1..12, -10..10	範囲型（整数の範囲制約）

型アノテーション構文

変数宣言時に型を明示できます。型が推論可能な場合は省略可能です。

```
let x: int = 42
let b: byte = 255
let f: float = 3.14
let s: str = "hello"
let b: bool = true
let opt: Option<int> = Some(10)
let t: (int, float) = (1, 3.14)
let xs: List<int> = [1, 2, 3]
let m: Map<str, int> = {"a": 1}
let s: Set<int> = {1, 2, 3}
let fn_val: fn(int) -> int = fn(x: int): x * 2
let u: int | str = 42
```

使用可能な型名一覧

型名	備考
int	組み込みスカラー型
byte	組み込みスカラー型（符号なし 0-255）
float	組み込みスカラー型
bool	組み込みスカラー型
str	組み込み文字列型
Unit	戻り値なし関数の戻り値型
Option<T>	ジェネリック型（T は任意の型）
(T1, T2, ...)	タプル型（要素数・型の組み合わせは任意）
List<T>	ジェネリック動的配列型
Map<K, V>	ジェネリックハッシュマップ型
Set<T>	ジェネリック集合型
fn(T1, ...) -> R	関数型
Error	組み込みエラー型（message: str、code: int）
T1 \ T2 \ ...	union 型（\ で区切った複数の型のいずれか）
ユーザー定義型名	record または enum キーワードで宣言した型
int リテラル型	int リテラル値による制約（例: 42, 0 \ 1）
str リテラル型	文字列リテラル値による制約（例: "N" \ "S"）
範囲型	整数の範囲による制約（例: 1..12, -10..10）

型エイリアス

type キーワードで既存の型に新しい名前を付けます。エイリアスは元の型と完全に互換性があります。

```
type Meters = float
type StringList = List<str>
```

```
let d: Meters = 3.14
let names: StringList = ["Alice", "Bob"]
```

命名規則: 型エイリアス名は PascalCase (例: Meters、StringList) を使用する必要があります。コンパイラがこの規則を強制します。

型エイリアスは関数型、リテラル型、範囲型にも使用できます:

```
type Callback = fn(int, int) -> int

let add: Callback = fn(a: int, b: int): a + b
print(add(3, 4))    # 7

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

let m: Month = 6
let d: Direction = "N"
let n: Digit = 5
```

リテラル型

リテラル型は、変数の値を特定の定数値に制限します。定数値の場合はコンパイル時に、動的な値の場合は実行時に制約がチェックされます。

int リテラル型

```
let x: 42 = 42           # 単一リテラル型
let y: 0 | 1 = 0         # int リテラルの union
var z: 0 | 1 = 0
z = 1                   # OK
# z = 2                 # コンパイルエラー (定数) または実行時エラー (動的値)
```

str リテラル型

```
let dir: "N" | "S" | "E" | "W" = "N"
# let bad: "N" | "S" = "X"    # コンパイルエラー
```

制約チェック

- **コンパイル時:** 代入値が定数 (ConstantInt や文字列リテラル) の場合、コンパイル時にチェックされ違反時はコンパイルエラー
- **実行時:** 値が動的 (関数の戻り値など) の場合、実行時にチェックされ違反時はプログラムがエラー終了

範囲型

範囲型は、整数変数の値を連続した範囲 (両端を含む) に制限します。

```
let month: 1..12 = 6      # OK
# let bad: 1..12 = 0      # コンパイルエラー: 範囲外
# let bad: 1..12 = 13     # コンパイルエラー: 範囲外
```

```
let t: -10..10 = -5          # 負の範囲もサポート
```

var での再代入（実行時チェック）

```
var x: 1..12 = 6
x = 12          # OK
# x = dynamic_value()  # 実行時チェック：範囲外ならエラー終了
```

関数パラメータでの使用

```
fn set_month(m: 1..12) -> int:
    return m

set_month(6)          # OK
# set_month(13)       # コンパイルエラー（定数引数）
```

none キーワードと Option 型の省略記法

none キーワードは Option 型の値が存在しないことを表し、None と同等です。

T? 構文は Option<T> の省略記法です。

```
let x: int? = 42          # Option<int> と同等
let y: int? = none        # None と同等
```

```
fn find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none
```

F-String（文字列補間）

f"..." 構文による文字列補間です。{} 内の式が評価され、文字列に変換されます。

```
let name = "world"
print(f"Hello {name}")    # Hello world

let a = 1
let b = 2
print(f"{a} + {b} = {a + b}")  # 1 + 2 = 3
```

補間で使用可能な型

{ } 内には int、float、bool、str に評価される任意の式を使用できます。

エスケープシーケンス

シーケンス	出力
{ {	{（リテラルの波括弧）
} }	}（リテラルの波括弧）
\n \t \\ \"	通常の文字列と同じ

```
print(f"{{{braces}}}")    # {braces}
```

型キャスト (as)

as キーワードによる明示的な型変換です。

```
let x = 42 as float      # 42.0
let y = 3.14 as int      # 3
let z = 1 as bool        # true
let s = 42 as str        # "42"
let b = 255 as byte      # byte 値 255
```

サポートされるキャスト

変換元	変換先	動作
int	float	SIToFP
float	int	切り捨て (FPToSI)
int	bool	0 -> false、非 0 -> true
bool	int	false -> 0、true -> 1
int / float / bool	str	文字列表現
int	byte	切り捨て (下位 8 ビット)
byte	int	ゼロ拡張

サポートされないキャスト (例: `str as int`) はコンパイルエラーになります。文字列から数値への変換には `to_int()` / `to_float()` を使用してください。

関連データを持つ enum (ADT)

バリエーション名の後ろに括弧で型を指定することで、enum バリエーションに関連データを持たせることができます。括弧なしのバリエーションは従来通りの単純なタグとして機能します。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

コンストラクタ

`EnumName::Variant(value)` の構文でデータ付きバリエーションを構築します。

```
let c = Shape::Circle(3.14)
let r = Shape::Rectangle(4.0, 5.0)
let p = Shape::Point
```

バインディング付きパターンマッチング

`case EnumName::Variant(binding):` の形式で関連データを取り出せます。

```
match c:
    case Shape::Circle(r):
        print(r)          # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")
```

内部表現

ADT enum はタグ付きユニオンとして格納されます: { i64 tag, [N x i8] data }。N は最大バリエントのペイロードに合わせたサイズです。

ジェネリック enum

enum は `<T>` の形式で型パラメータを持てます。これにより、同じ enum 構造で異なる型のペイロードを保持できます。

```
enum MyOption<T>:  
    MySome(T)  
    MyNone
```

使用法

コンパイラが型を推論できない場合は、具体的な型引数を指定してインスタンス化します。

```
let a = MyOption<int>::MySome(42)  
let b = MyOption<int>::MyNone
```

```
match a:  
    case MyOption::MySome(v):  
        print(v)           # 42  
    case MyOption::MyNone:  
        print("none")
```

Error 型

エラーハンドリング用の組み込み型です。Error は message (str) と code (int) の 2 つのフィールドを持ちます。

```
let e = Error("something went wrong")           # code のデフォルトは 0  
let e2 = Error("not found", 404)                # 明示的な code  
  
print(e.message)    # something went wrong  
print(e2.code)       # 404  
print(e2)            # Error: not found (code: 404)
```

エラーハンドリング規約

失敗する可能性のある関数は (T, Error?) タプルを返します:

```
fn divide(a: int, b: int) -> (int, Error?):  
    if b == 0:  
        return (0, Some(Error("division by zero")))  
    return (a // b, none)
```

```
let val, err = divide(10, 2)  
match err:  
    case Some(e):  
        print(e.message)  
    case None:  
        print(val)           # 5
```

!! 演算子（エラー伝播）

!! 後置演算子は (T, Error?) タプルから値を取り出します。エラーが存在する場合、そのエラーは囲む関数に伝播されます。

```
fn read_file(path: str) -> (str, Error?):
    if path == "":
        return ("", Some(Error("empty path")))
    return ("content", none)

fn process() -> (str, Error?):
    let data = read_file("test.txt")!!    # エラーがあれば伝播
    return (data, none)
```

囲む関数も (X, Error?) を返す必要があります。

内部表現

Error は { ptr message, i64 code } として表現されます。

union 型

| を使って複数の型を持ちうる変数を宣言できます。

```
let x: int | str = 42
x = "hello"      # 再代入可能 (union のいずれかの型)
print(x)         # hello
```

関数引数・戻り値での使用

```
fn show(x: int | str) -> int:
    print(x)
    return 0

fn get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

内部表現

union 型は { i64 tag, [N x i8] data } として表現されます。tag は各コンポーネント型のインデックス（アルファベット順ソート後）を示し、data は最大コンポーネントサイズ分のバイト配列です。

制約

- union に含まれない型を代入するとコンパイルエラー
- int | str と str | int は同じ型（正規化される）
- print() で union 値を出力すると、実行時のタグに基づいて適切な型で表示される

型規則（演算時の型変換）

演算	左辺	右辺	結果型	備考
+ - *	int	int	int	
+ - *	byte	byte または int	int	byte は演算時に int へ ZExt 昇格

演算	左辺	右辺	結果型	備考
+ - *	float または int	float または int (片方が float)	float	暗黙の float 昇格
/	任意の数値	任意の数値	float	常に float
//	任意の数値	任意の数値	int	float 入力は切り捨て変換
**	任意の数値	任意の数値	float	libm pow 使用
%	int	int	int	
%	float または int	float または int (片方が float)	float	
+	str	str	str	文字列結合
== != < <= > >=	str	str	bool	辞書順比較
== != < <= > >=	数値または bool	数値または bool	bool	
in	任意	Set	bool	要素がセットに含まれるか
& \ ^ ~ << >>	int	int	int	float にはエラー

エスケープシーケンス (str リテラル内)

シーケンス	意味
\n	改行
\t	タブ
\\	バックスラッシュ
\"	ダブルクォート
\0	ヌル文字

型安全性の制約

- 暗黙の型変換はない — int と float を混在させると float への昇格が発生するが、それ以外の暗黙変換は存在しない。byte は演算時に int へ自動昇格する (ZExt)。型アノテーション `let b: byte = 42` でのみ int リテラルから byte への縮小変換が許可される。
- 変数の型は宣言時に固定される — 一度 int として宣言した変数に float を再代入することはできない。
- ビット演算は int のみ — float や bool に対してビット演算を適用するとコンパイルエラー。
- bool 以外の型も条件式に使える — if の条件式には int (0 = false、非 0 = true) など bool 以外も使用可能。